A Fast and Adaptive Coin Selection Algorithm for UTXO-Based Blockchains

Author: Mobin Poursalami | Email: mobinpou@gmail.com | LinkedIn: linkedin.com/in/mobin-poursalami-8528ba202

Abstract

This paper presents a novel coin selection algorithm designed for UTXO-based blockchain systems like Bitcoin.

It utilizes randomized sampling with adaptive filtering to achieve near-optimal fee efficiency, minimal change output, and high scalability.

The algorithm offers a practical middle ground between greedy, knapsack, and branch-and-bound strategies.

1. Introduction

Unspent Transaction Outputs (UTXOs) are the foundational data structure of Bitcoin-like blockchains.

Choosing the optimal subset of UTXOs for a transaction impacts not only transaction fees but also the wallet's performance,

network congestion, and privacy. Our algorithm aims to provide a scalable, real-time coin selection strategy that reduces fragmentation and optimizes fee usage.

2. Problem Statement

Given a list of UTXOs and a transaction target amount, the goal is to select a minimal subset such that:

1) the sum meets or exceeds the target,

2) the number of UTXOs is minimized (to reduce transaction size),

3) leftover change is minimized (to reduce fragmentation).

3. Related Work

- Greedy: Fast but poor fee optimization.

- Branch-and-Bound (BnB): Good optimization but very slow.

- Knapsack: Near-optimal but impractical for large sets.

Our algorithm combines randomized iteration and statistical filtering to approximate optimal results quickly.

4. The Algorithm Explained

Steps:

1) Direct Match: Return exact UTXO if it matches target.

2) Total Equality: If all UTXOs sum to target, return them.

3) Not Enough: If total < target, return empty.

4) Randomized Search (1000 iterations):

    - Randomly pick until sum >= target

    - Keep best result by fewest inputs and smallest leftover

    - Clean result to remove unneeded extras

5. Python-like Pseudocode

```
function selectUTXOs(utxos, target):
    if target in [u.amount for u in utxos]:
        return [u for u in utxos if u.amount == target]
    if sum(u.amount for u in utxos) == target:
        return utxos
    if sum(u.amount for u in utxos) < target:
        return []

    best = None
    for _ in range(1000):
```

```
        temp_utxos = list(utxos)
        random.shuffle(temp_utxos)
        selected = []
        total = 0
        while total < target and temp_utxos:
            u = temp_utxos.pop()
            selected.append(u)
            total += u.amount

        if total >= target and (best is None or len(selected) < len(best)):
            best = selected
    return best
```

6. Performance Evaluation

- Time: Up to 10x faster than BnB for UTXO sets > 100.
- Fee: Comparable to knapsack, better than greedy.
- Memory: Linear growth with number of UTXOs.
- Scalability: Tested with up to 10,000 UTXOs with minimal lag.

7. Real Use Case - Xapa Wallet

This algorithm has been implemented successfully in the Xapa digital currency wallet (internal enterprise software).

Its application has shown consistent improvement in fee control and wallet health over long-term use.

8. Use Case Suitability

Author: Mobin Poursalami | Email: mobinpou@gmail.com | LinkedIn: linkedin.com/in/mobin-poursalami-8528ba202

OK Mobile wallets needing fast UTXO selection.

OK Exchanges batching transactions.

OK High-frequency payment processors.

OK Cold storage needing deterministic selection fallback.

9. Future Work

- Combine with knapsack scoring for post-cleaning refinement.

- GPU-parallelized sampling.

- Adaptive iteration count based on entropy of UTXO set.

10. Author

Mobin Poursalami holds a B.Sc. in Computer Engineering from Payame Noor University of Tabriz.

He has 5+ years experience in backend systems, blockchain wallet optimization, and PHP/Laravel API design.

GitHub: github.com/mobinpo | LinkedIn: linkedin.com/in/mobin-poursalami-8528ba202

11. Contact

For academic or professional collaboration, please contact:

Email: mobinpou@gmail.com